

TÀI LIỆU THIẾT KẾ HỆ THỐNG

Blue-Green Deployment cho Schema Migration: Zero-Downtime Update

Đồ án số 127 – Nhóm hirono – Vương Minh Thắng – Học kỳ II-2026

1. Đặt vấn đề

Trong môi trường production của hệ thống e-commerce, việc nâng cấp schema database từ phiên bản V1 cũ sang V2 mới thường gây ra downtime đáng kể do migration kiểu blocking yêu cầu khóa bảng, sự gắn kết chặt giữa code và schema, và khó khăn khi rollback nếu phát hiện bug ở schema mới. Đồ án này triển khai Blue-Green Deployment Pattern kết hợp với Lazy Distributed Replication (Özsu & Valduriez, Chương 6) để đạt được mục tiêu zero-downtime schema migration. Hai cluster MySQL chạy song song với hai schema khác nhau, được kết nối thông qua một Synchronization Bridge tự xây dựng — thực hiện bidirectional asynchronous replication và schema translation V1 ↔ V2 trong thời gian thực.

2. Kiến trúc tổng thể

Hệ thống bao gồm ba Spring Boot services và ba thành phần hạ tầng, được deploy trên Kubernetes cluster (minikube cho môi trường demo):

Service	Port	Vai trò
Blue Service	8081	REST API trên schema V1 (cũ, id BIGINT, denormalized)
Green Service	8082	REST API trên schema V2 (mới, id UUID, normalized với FK)
Sync Bridge	8083	Bidirectional async replicator + Schema Translator V1↔V2
MySQL Blue	3307	Database catalog_v1 + bảng outbox_events local
MySQL Green	3308	Database catalog_v2 + bảng outbox_events local
RabbitMQ	5672	Retry queue cho các replication bị thất bại

Giao tiếp giữa client và service sử dụng HTTP/REST. Sync Bridge thực hiện polling-based Change Data Capture: poll bảng outbox_events của mỗi cluster mỗi 2 giây, dịch payload V1 ↔ V2 thông qua Schema Translator, rồi gọi POST/PUT/DELETE qua REST sang service đích.

3. Các quyết định thiết kế chính

Quyết định thiết kế	Lý do lựa chọn	Tham chiếu Özsu 4th Ed.
Lazy (async) replication thay vì Eager (2PC)	Eager 2PC bị block khi có failure, vi phạm yêu cầu zero-downtime; Lazy đảm bảo high availability với eventual consistency	Ch.6 §6.3.3, §6.3.4
Outbox Pattern (transactional outbox)	Đảm bảo atomic dual-write: ghi business data và event publish trong cùng 1 DB transaction, không mất dữ liệu khi crash	Ch.6 §6.3.3
Schema Mapping	Cho phép 2 cluster với schema khác nhau	Ch.7 §7.1.4

Layer (V1 ↔ V2)	cùng tồn tại; áp dụng kỹ thuật multidatabase integration	
Loop prevention dựa trên origin tag	Gắn tag origin (BLUE/GREEN) vào event; bỏ qua event không có nguồn gốc từ cluster local, chống vòng lặp vô tận	Ch.6 §6.3.4
Last-Write-Wins conflict resolution	Đơn giản, deterministic dựa trên timestamp updated_at; phù hợp với product catalog domain	Ch.6 §6.3.4
Polling-based CDC thay vì trigger-based	Tách biệt logic sync khỏi database; xử lý lỗi đơn giản; dễ test; không phụ thuộc DBMS specific feature	Ch.6 §6.5

4. Chi tiết Schema Mapping V1 → V2

Bảng dưới đây tóm tắt sáu phép biến đổi schema được Schema Translator xử lý:

V1 (Blue)	V2 (Green)	Phép biến đổi
id BIGINT	id UUID	Đổi kiểu, lưu mapping ở bảng id_mapping
price DECIMAL	price_amount + currency	Tách field, mặc định USD
category VARCHAR	category_id BIGINT FK	Normalize vào bảng categories
brand VARCHAR	brand_id BIGINT FK	Normalize vào bảng brands
availability_status VARCHAR	availability_status ENUM	Chuyển free-text thành enum chuẩn
(không có)	dimensions JSON, metadata JSON	Field mới chỉ có ở V2

5. Công nghệ sử dụng

Thành phần	Công nghệ sử dụng
Ngôn ngữ	Java 17
Framework	Spring Boot 3.2 (Web, Data JPA, Validation)
Cơ sở dữ liệu	MySQL 8.0 (2 instances độc lập, không share)
Migration	Liquibase XML changelogs
Container	Docker Compose (dev) + Kubernetes minikube (production)
Message Queue	RabbitMQ 3 (retry queue cho replication thất bại)
Build Tool	Maven multi-module (parent + 3 sub-modules)

6. Chiến lược xử lý lỗi (Failure Handling Strategy)

Hệ thống được thiết kế để xử lý năm kịch bản failure phổ biến, đảm bảo eventual consistency và zero-downtime:

Pod failure (kill Green/Blue): Kubernetes Deployment Controller tự động restart pod thông qua replica set; bảng outbox đảm bảo các event đang chờ không bị mất, được replicate khi pod hồi phục.

Sync Bridge failure: các event vẫn nằm an toàn trong outbox của cluster nguồn, được @Scheduled poller pickup lại khi Bridge khởi động trở lại.

Network partition: Blue và Green vẫn tiếp tục nhận write request bình thường; outbox tích lũy event theo thứ tự thời gian, được catch-up đầy đủ khi mạng phục hồi.

Concurrent conflict: Last-Write-Wins resolution dựa trên timestamp updated_at; cả hai write đều thành công và dữ liệu hội tụ về eventual consistency.

Rollback scenario: scale down Green về 0 replicas khi phát hiện V2 có bug; Blue tiếp tục phục vụ traffic; khi Green online trở lại, Sync Bridge tự động tiếp tục replicate.

7. Tài liệu tham khảo

Özsu, M. T., & Valduriez, P. (2020). *Principles of Distributed Database Systems, 4th Edition*. Springer Nature.

Chương 6: Data Replication — cơ sở lý thuyết cho Lazy Replication, Outbox Pattern, conflict resolution

Chương 7: Database Integration — Multidatabase Systems — kỹ thuật Schema Mapping V1 ↔ V2

Chương 5: Distributed Transaction Processing — nguyên tắc atomic transaction và reliability